

Variational Perspective: From VAEs to DDPMs – I

March 2026

<https://linklab.github.io>

Variational Autoencoder

Basic Concepts of Variational Autoencoders

! Limitations of Standard Autoencoders

- ⊗ Deterministic encoding produces **unstructured latent space** (random latent codes)
- ⊗ Random latent codes typically produce **meaningless outputs**
- ⊗ Limited use for generation tasks

✓ VAE's Probabilistic Approach

- Imposes **probabilistic structure** on latent space
- Stochastic encoder $q_{\theta}(\mathbf{z} | \mathbf{x})$ and decoder $p_{\phi}(\mathbf{x} | \mathbf{z})$
- Latent space becomes **smooth and continuous**
- Foundation for Advanced Models

VAEs: **true generative models**

VAEs

HVAEs

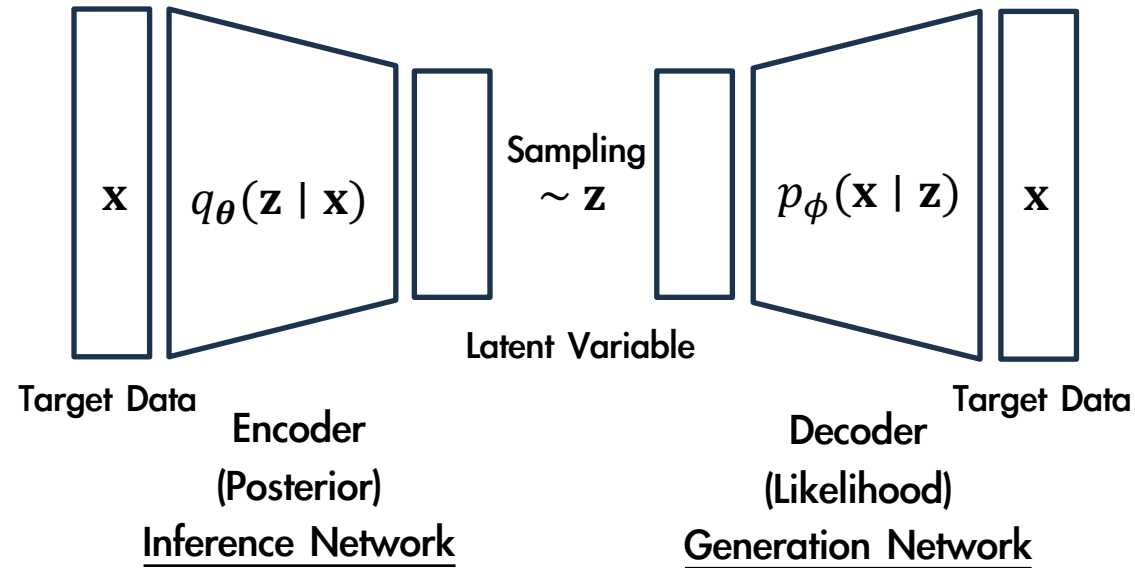
DDPMs



Evolution →

VAE framework extends to **hierarchical models** (HVAEs) and **diffusion models** (DDPMs)

Probabilistic Encoder and Decoder in VAEs



⚙️ Prior Distribution

$$\mathbf{z} \sim p_{\text{prior}} = \mathcal{N}(0, 1)$$

- Latent variables follow standard Gaussian prior

⚙️ Decoder Distribution

$$p_{\phi}(\mathbf{x} | \mathbf{z})$$

- Maps latent variables to data space

[Key Relationships in VAEs]

↻ **Probabilistic Encoding**
 $q_{\theta}(\mathbf{z} | \mathbf{x})$ maps data to latent

↻ **Probabilistic Decoding**
 $p_{\phi}(\mathbf{x} | \mathbf{z})$ generates data from latent

↻ **Latent–Data Relationship**
 Connects observed and latent variables

VAE Decoder Construction and Marginal Likelihood

⚙️ Latent-Variable Generative Model

The VAE defines a latent-variable generative model through the marginal likelihood:

$$p_{\phi}(\mathbf{x}) = \int p_{\phi}(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

x가 나올 확률은, 모든 가능한 z가 x를 만들 확률을 합친 것

- > Observed data $\mathbf{x}$ generated from latent variable $\mathbf{z}$
- > Latent prior $p(\mathbf{z})$ typically standard Gaussian

Ideally $\max_{\phi} \log p_{\phi}(\mathbf{x})$



⚠️ Computational Challenge

- ✖ The decoder parameters ϕ at $p_{\phi}(\mathbf{x} | \mathbf{z})$ are learned by MLE
- ✖ The integral over $\mathbf{z}$ is intractable
- ✖ Direct MLE at $p_{\phi}(\mathbf{x})$ is computationally infeasible

💡 Variational Approach

[Reverse question] given an observation $\mathbf{x}$, what latent codes $\mathbf{z}$ could have produced it?

√x Posterior

$$p_{\phi}(\mathbf{z} | \mathbf{x}) = \frac{p_{\phi}(\mathbf{x} | \mathbf{z}) p(\mathbf{z})}{p_{\phi}(\mathbf{x})} \leftarrow$$

- By Bayes' rule, the posterior distribution over latent variables $\mathbf{z}$ given observation $\mathbf{x}$
- The denominator $p_{\phi}(\mathbf{x})$ still requires integrating over all latent variables

데이터 x에 대해, 우리가 궁금한 것 → 이 x는 대체 어떤 z에서 생성될 것일까? → Posterior Distribution → 학습할 때는 Posterior를 통하여 x를 잘 설명하는 z가 무엇인지 알아야 decoder가 올바르게 학습됨

To address the computational infeasibility, we introduce an encoder (or inference network) $q_{\theta}(\mathbf{z} | \mathbf{x})$, whose role is to serve as a learnable approx. posterior and replace the intractable posterior:

$$q_{\theta}(\mathbf{z} | \mathbf{x}) \approx p_{\phi}(\mathbf{z} | \mathbf{x})$$

In practice, $q_{\theta}(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mu_{\theta}(\mathbf{x}), \sigma_{\theta}(\mathbf{x}))$

Evidence Lower Bound (ELBO) Derivation

Theorem 2.1.1: Evidence Lower Bound (ELBO)

For any data point $\mathbf{x}$, the log-likelihood satisfies:

$$\log p_{\phi}(\mathbf{x}) \geq \mathcal{L}_{\text{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x})$$

where the ELBO is given by:

$$\mathcal{L}_{\text{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})} [\log p_{\boldsymbol{\phi}}(\mathbf{x} | \mathbf{z})]}_{\text{Reconstruction Term}} - \underbrace{\mathcal{D}_{\text{KL}}(q_{\boldsymbol{\theta}}(\mathbf{z} | \mathbf{x}) \| p_{\text{prior}}(\mathbf{z}))}_{\text{Latent Regularization}} \quad [2.1.1]$$

[Proof] $\log p_{\phi}(\mathbf{x}) = \log \int p_{\phi}(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \log \int q_{\theta}(\mathbf{z} | \mathbf{x}) \frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{q_{\theta}(\mathbf{z} | \mathbf{x})} d\mathbf{z}$

$$= \log \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} \left[\frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{q_{\theta}(\mathbf{z} | \mathbf{x})} \right] \geq \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{q_{\theta}(\mathbf{z} | \mathbf{x})} \right] = \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{z}) p_{\phi}(\mathbf{x} | \mathbf{z})}{q_{\theta}(\mathbf{z} | \mathbf{x})} \right]$$

$$= \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} [p_{\phi}(\mathbf{x} | \mathbf{z})] + \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{z}) - \log q_{\theta}(\mathbf{z} | \mathbf{x})]$$

$$= \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} [p_{\phi}(\mathbf{x} | \mathbf{z})] - \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} [\log q_{\theta}(\mathbf{z} | \mathbf{x}) - \log p(\mathbf{z})]$$

$$= \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} [\log p_{\phi}(\mathbf{x} | \mathbf{z})] - \mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))$$

Evidence Lower Bound (ELBO) Derivation

Theorem 2.1.1: Evidence Lower Bound (ELBO)

For any data point $\mathbf{x}$, the log-likelihood satisfies:

$$\log p_{\phi}(\mathbf{x}) \geq \mathcal{L}_{\text{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}, \mathbf{x})$$

where the ELBO is given by:

$$\mathcal{L}_{\text{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})} [\log p_{\boldsymbol{\phi}}(\mathbf{x} | \mathbf{z})]}_{\text{accurate reconstruction of the data}} - \underbrace{\mathcal{D}_{\text{KL}}(q_{\boldsymbol{\theta}}(\mathbf{z} | \mathbf{x}) \| p_{\text{prior}}(\mathbf{z}))}_{\text{regularizing the latent variables by keeping them close to a simple prior distribution } p_{\text{prior}}(\mathbf{z}) \text{ (often Gaussian)}} \quad [2.1.1]$$

accurate reconstruction of the data

regularizing the latent variables by keeping them close to a simple prior distribution $p_{\text{prior}}(\mathbf{z})$ (often Gaussian)

- 각각의 $\mathbf{z}$ 로부터 $\mathbf{x}$ 를 최대한 정확히 복원
- 각 데이터 $\mathbf{x}$ 마다 서로 다른 $\mathbf{z}$ 산출 노력

$\mathbf{z}$ 가 가능한 정보를 많이 담도록 노력

- 각 데이터 $\mathbf{x}$ 마다 산출되는 $\mathbf{z}$ 의 정보는 $\mathcal{N}(0, 1)$ 분포를 지니도록 제한

$\mathbf{z}$ 가 가능한 정보를 압축하여 적게 담도록 노력

Trade-off

Key Insight

ELBO provides a tractable lower bound on log-likelihood, enabling optimization via gradients.

항목	Reconstruction	Latent KL
$\mathbf{z}$ 역할	정보 저장	분포 정규화
$\mathbf{x}$ 간 구분	강하게	약하게
분산	작게	크게
평균	자유롭게	0 근처

Information-Theoretic View: ELBO as a Divergence Bound

True modeling error: Original MLE Problem

Minimizing $\mathcal{D}_{\text{KL}}(p_{\text{data}}(\mathbf{x}) \| p_{\phi}(\mathbf{x}))$

Two Joint Distributions

- > Generative joint: $p_{\phi}(x, z) = p(z)p_{\phi}(x | z)$
- > Inference joint: $q_{\theta}(x, z) = p_{\text{data}}(x)q_{\theta}(z | x)$

Comparing these distributions yields

$$\mathcal{D}_{\text{KL}}(p_{\text{data}}(x) \| p_{\phi}(x)) \leq \mathcal{D}_{\text{KL}}(q_{\theta}(x, z) \| p_{\phi}(x, z)) \quad [2.1.2]$$

sometimes referred to as **the chain rule for KL divergence**

Also, note that

$$\log p_{\phi}(\mathbf{x}) - \mathcal{L}_{\text{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} | \mathbf{x}) \| p_{\phi}(\mathbf{z} | \mathbf{x}))$$

Joint KL divergence expands as:

$$\begin{aligned} \mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{x}, \mathbf{z}) \| p_{\phi}(\mathbf{x}, \mathbf{z})) \\ = \mathcal{D}_{\text{KL}}(p_{\text{data}}(\mathbf{x}) \| p_{\phi}(\mathbf{x})) + \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} | \mathbf{x}) \| p_{\phi}(\mathbf{z} | \mathbf{x}))] \end{aligned}$$

✓ True modeling error: $\mathcal{D}_{\text{KL}}(p_{\text{data}}(x) \| p_{\phi}(x))$

✓ Inference error: $\mathbb{E}_{p_{\text{data}}(x)} [\mathcal{D}_{\text{KL}}(q_{\theta}(z | x) \| p_{\phi}(z | x))]$

💡 Key Insight

Maximizing the ELBO corresponds to reducing inference error, ensuring training minimizes a meaningful part of the overall bound.

Gaussian VAE Implementation

Encoder

Conditional distribution:

$$q_{\theta}(\mathbf{z} \mid \mathbf{x}) := \mathcal{N}\left(\mathbf{z}; \boldsymbol{\mu}_{\theta}(\mathbf{x}), \text{diag}\left(\boldsymbol{\sigma}_{\theta}^2(\mathbf{x})\right)\right)$$

- > Maps observed data $\mathbf{x}$ to latent code $\mathbf{z}$
- > Gaussian distribution with learned parameters
- > Encoder parameters:

$$\boldsymbol{\mu}_{\theta}: \mathbb{R}^D \rightarrow \mathbb{R}^d \text{ and } \boldsymbol{\sigma}_{\theta}: \mathbb{R}^D \rightarrow \mathbb{R}_+^d$$

Decoder

Conditional distribution:

$$p_{\phi}(\mathbf{x} \mid \mathbf{z}) := \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_{\phi}(\mathbf{z}), \sigma^2 \mathbf{I})$$

- > Maps latent code $\mathbf{z}$ back to data $\mathbf{x}$
- > Fixed small variance σ
- > Decoder parameters: $\boldsymbol{\mu}_{\phi}: \mathbb{R}^d \rightarrow \mathbb{R}^D$
- > **Output:** Reconstruction $\mathbf{x} = \boldsymbol{\mu}_{\phi}(\mathbf{z})$,
where $\mathbf{z} \sim q_{\theta}(\mathbf{z} \mid \mathbf{x})$

Under this assumption,
the reconstruction term in the ELBO simplifies as

$$\mathbb{E}_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log p_{\phi}(\mathbf{x} \mid \mathbf{z})] = -\frac{1}{2\sigma^2} \mathbb{E}_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\|\mathbf{x} - \boldsymbol{\mu}_{\phi}(\mathbf{z})\|^2] + C$$

→ C is a constant independent of θ and ϕ

The ELBO objective $\min_{\theta, \phi} \mathbb{E}_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \left[\frac{1}{2\sigma^2} \|\mathbf{x} - \boldsymbol{\mu}_{\phi}(\mathbf{z})\|^2 \right] + \mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\text{prior}}(\mathbf{z})),$
a closed-form solution

VAE's Blurry Generation Problem

⚙️ VAE Problem Reformulated

Consider a fixed Gaussian encoder and a decoder

$$q_{\theta}(\mathbf{z} | \mathbf{x}) := \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_{\theta}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_{\theta}^2(\mathbf{x})))$$

$$p_{\phi}(\mathbf{x} | \mathbf{z}) := \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_{\phi}(\mathbf{z}), \sigma^2 \mathbf{I})$$

Optimizing the ELBO reduces to minimizing the expected reconstruction error

$$\arg \min_{\boldsymbol{\mu}} \mathbb{E}_{p_{\text{data}}(\mathbf{x}) q_{\text{enc}}(\mathbf{z} | \mathbf{x})} [\|\mathbf{x} - \boldsymbol{\mu}(\mathbf{z})\|^2]$$

데이터 $\mathbf{x}$ 를 encoder로 $\mathbf{z}$ 를 만든 다음, decoder $\boldsymbol{\mu}(\mathbf{z})$ 로 복원했을 때 $\mathbf{x}$ 와의 오차를 최소화하는 decoder $\boldsymbol{\mu}$ 를 찾아라.

[Solution]

This is a standard least squares problem in $\boldsymbol{\mu}(\mathbf{z})$, and its solution is given in closed form

$$\boldsymbol{\mu}^*(\mathbf{z}) = \mathbb{E}_{q_{\text{enc}}(\mathbf{x} | \mathbf{z})}[\mathbf{x}]$$

$q_{\text{enc}}(\mathbf{x} | \mathbf{z})$ is the encoder-induced posterior on inputs given latents

$$q_{\text{enc}}(\mathbf{x} | \mathbf{z}) = \frac{q_{\text{enc}}(\mathbf{z} | \mathbf{x}) p_{\text{data}}(\mathbf{x})}{p_{\text{prior}}(\mathbf{z})}$$

$$\boldsymbol{\mu}^*(\mathbf{z}) = \frac{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\text{enc}}(\mathbf{z} | \mathbf{x}) \cdot \mathbf{x}]}{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\text{enc}}(\mathbf{z} | \mathbf{x})]}$$

VAE's Blurry Generation Problem

[증명] $\mu^*(\mathbf{z}) = \mathbb{E}_{q_{\text{enc}}(\mathbf{x}|\mathbf{z})}[\mathbf{x}]$

$$= \int \mathbf{x} \cdot q_{\text{enc}}(\mathbf{x}|\mathbf{z}) d\mathbf{x}$$

$$= \int \mathbf{x} \cdot \frac{q_{\text{enc}}(\mathbf{z}|\mathbf{x})p_{\text{data}}(\mathbf{x})}{p_{\text{prior}}(\mathbf{z})} d\mathbf{x}$$

$$= \frac{1}{p_{\text{prior}}(\mathbf{z})} \int \mathbf{x} \cdot q_{\text{enc}}(\mathbf{z}|\mathbf{x})p_{\text{data}}(\mathbf{x}) d\mathbf{x}$$

$$= \frac{\int q_{\text{enc}}(\mathbf{z}|\mathbf{x})p_{\text{data}}(\mathbf{x}) \cdot \mathbf{x} d\mathbf{x}}{\int q_{\text{enc}}(\mathbf{z}|\mathbf{x})p_{\text{data}}(\mathbf{x}) d\mathbf{x}}$$

$$= \frac{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\text{enc}}(\mathbf{z}|\mathbf{x}) \cdot \mathbf{x}]}{\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[q_{\text{enc}}(\mathbf{z}|\mathbf{x})]}$$

Dataset: MNIST 이미지 3장

[구체적인 예시]

- $\mathbf{x}_1$: "1", $\mathbf{x}_2$: "7", $\mathbf{x}_3$: "9"

Latent point: $\mathbf{z}^* = [0.5, 0.3]$

Encoder 확률:

- $q_{\text{enc}}(\mathbf{z}^*|\mathbf{x}_1) = 0.1$
- $q_{\text{enc}}(\mathbf{z}^*|\mathbf{x}_2) = 0.3$
- $q_{\text{enc}}(\mathbf{z}^*|\mathbf{x}_3) = 0.05$

최적 Decoder 계산

$$\mu^*(\mathbf{z}^*) = \frac{0.1 \cdot \mathbf{x}_1 + 0.3 \cdot \mathbf{x}_2 + 0.05 \cdot \mathbf{x}_3}{0.1 + 0.3 + 0.05} = 0.22 \cdot \mathbf{x}_1 + 0.67 \cdot \mathbf{x}_2 + 0.11 \cdot \mathbf{x}_3$$

결과: "1" (22%) + "7" (67%) + "9" (11%) = 블러

VAE's Blurry Generation Problem

Now suppose that two distinct inputs $x \neq x'$ are mapped to overlapping regions in latent space, i.e., the supports of $q_{enc}(\cdot | x)$ and $q_{enc}(\cdot | x')$ intersect.

Then, $\mu^*(z)$ averages over multiple, potentially unrelated inputs, which leads to blurry, non-distinct outputs.

$$\mu^*(z) = \frac{\mathbb{E}_{p_{data}(x)}[q_{enc}(z | x) \cdot x]}{\mathbb{E}_{p_{data}(x)}[q_{enc}(z | x)]}$$

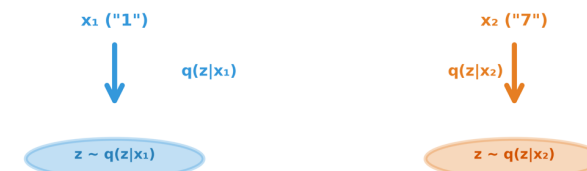
This averaging effect over conflicting modes is a fundamental reason for the characteristic blurriness in VAE-generated samples.

Step 1: Input Space



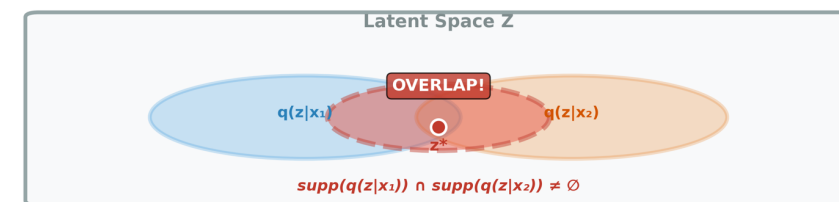
Two distinct inputs: "1" and "7"

Step 2: Encoding to Latent Space



Each input is encoded to a Gaussian distribution in latent space

Step 3: Overlapping Support in Latent Space

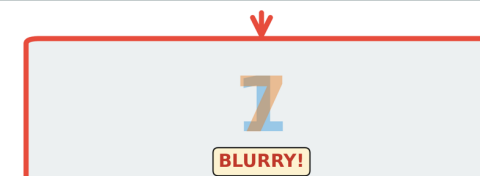


The supports of the two distributions overlap!
Both inputs contribute to point z^*

Step 4: Decoder Output at z^*

Optimal Decoder (Weighted Average):

$$\mu^*(z^*) = w_1 \cdot x_1 + w_2 \cdot x_2$$



Average of "1" and "7" $\neq$ Any real digit $\rightarrow$ Unrealistic output

Why Deeper Networks in a Flat VAE are Not Enough

⚙️ Limit of Posterior Family

$$q_{\theta}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\left(\mathbf{z}; \boldsymbol{\mu}_{\theta}(\mathbf{x}), \text{diag}\left(\sigma_{\theta}^2(\mathbf{x})\right)\right)$$

- It is a single Gaussian with diagonal covariance
- Deeper network can improve the accuracy of $\boldsymbol{\mu}_{\theta}$ and σ_{θ} but does not expand the family

$$q_{\theta}(\mathbf{z} \mid \mathbf{x}) \approx p_{\phi}(\mathbf{z} \mid \mathbf{x})$$

- When $p_{\phi}(\mathbf{z} \mid \mathbf{x})$ is multi-peaked (or multi-modal), $q_{\theta}(\mathbf{z} \mid \mathbf{x})$ cannot match it
- It loosens the ELBO and weakens inference.
- Addressing this requires a richer posterior class, not merely deeper networks

Why Deeper Networks in a Flat VAE are Not Enough

⚙️ Posterior Collapse

- if the decoder is too expressive, the model may suffer from posterior collapse
- In order to explain posterior collapse, we should consider all samples $\sim p_{\text{data}}$

$$\begin{aligned}\mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\mathcal{L}_{\text{ELBO}}(\mathbf{x})] \\ &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})q_{\theta}(\mathbf{z}|\mathbf{x})}[\log p_{\phi}(\mathbf{x}|\mathbf{z})] - \mathbb{E}_{p_{\text{data}}(\mathbf{x})}[\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))] \\ &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})q_{\theta}(\mathbf{z}|\mathbf{x})}[\log p_{\phi}(\mathbf{x}|\mathbf{z})] - \mathcal{I}_q(\mathbf{x}; \mathbf{z}) - \mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z})\|p(\mathbf{z})),\end{aligned}$$

[posterior collapse]

- 모든 입력 $\mathbf{x}$ 가 동일한 $\mathbf{z}$ 분포로 매핑되는 현상
- $q_{\theta}(\mathbf{z}|\mathbf{x}) \approx q_{\theta}(\mathbf{z}) \approx p(\mathbf{z})$ for all $\mathbf{x}$

where $\mathcal{I}_q(\mathbf{x}; \mathbf{z})$ is the mutual information defined by

$$\mathcal{I}_q(\mathbf{x}; \mathbf{z}) = \mathbb{E}_{q(\mathbf{x}, \mathbf{z})} \left[\log \frac{q_{\theta}(\mathbf{z} | \mathbf{x})}{q(\mathbf{z})} \right] = \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} | \mathbf{x}) \| q(\mathbf{z}))]$$

and the aggregated posterior $q(\mathbf{z})$ is $q_{\theta}(\mathbf{z}) = \int p_{\text{data}}(\mathbf{x})q_{\theta}(\mathbf{z} | \mathbf{x})d\mathbf{x}$

- a maximizer of the ELBO tries set $\mathcal{I}_q(\mathbf{x}; \mathbf{z}) = 0$ and $q_{\theta}(\mathbf{z}) = p(\mathbf{z})$.
- This "ignore $\mathbf{z}$ (posterior collapse)" solution does not disappear by making the networks deeper

From Standard VAE to Hierarchical VAEs

Hierarchical Variational Autoencoders (HVAEs)

⚙️ HVAEs Summary

- Purpose: Extend VAE hierarchically for complex data modeling
- Structure: Introduce multiple layers of latent variable hierarchy
- Advantage: Capture data features at multiple abstraction levels → significantly boost expressive power
- Characteristic: Naturally mirror the compositional nature of real-world data

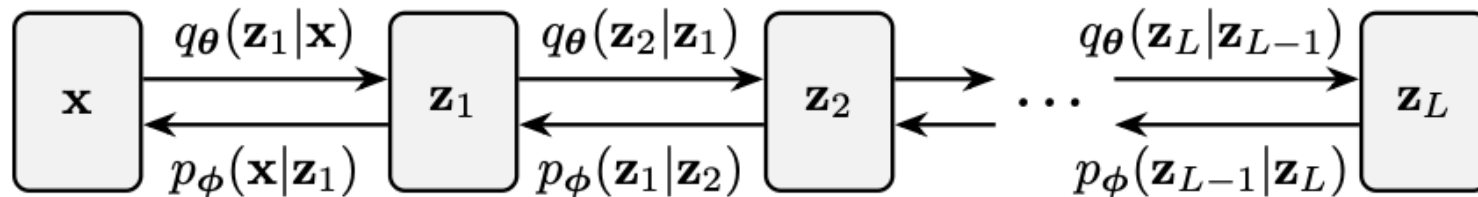


Figure 2.2: Computation graph of the HVAE. It has a hierarchical structure with stacked, trainable encoders and decoders across multiple latent layers.

HVAE's Modeling

⚙️ HVAEs Decoder Structure

- Architecture: Multiple latent layers
 $p_\phi(z_{i-1} | z_i)$ in top-down hierarchy
- Chain: Each layer conditions the next, forming conditional priors
- Abstraction: Progressively finer levels (coarse $\rightarrow$ fine)

This leads to the following top-down factorization of the joint distribution

$$p_\phi(\mathbf{x}, \mathbf{z}_{1:L}) = p_\phi(\mathbf{x} | \mathbf{z}_1) \prod_{i=2}^L p_\phi(\mathbf{z}_{i-1} | \mathbf{z}_i) p(\mathbf{z}_L).$$



This structure defines the marginal data distribution

$$p_{\text{HVAE}}(\mathbf{x}) := \int p_\phi(\mathbf{x}, \mathbf{z}_{1:L}) d\mathbf{z}_{1:L}$$

⚙️ HVAE Encoder Structure

Learnable encoder $q_\theta(\mathbf{z}_{1:L} | \mathbf{x})$ mirrors generative hierarchy, typically using bottom-up Markov factorization

$$q_\theta(\mathbf{z}_{1:L} | \mathbf{x}) = q_\theta(\mathbf{z}_1 | \mathbf{x}) \prod_{i=2}^L q_\theta(\mathbf{z}_i | \mathbf{z}_{i-1})$$

Bottom (Down)

Top (Up)

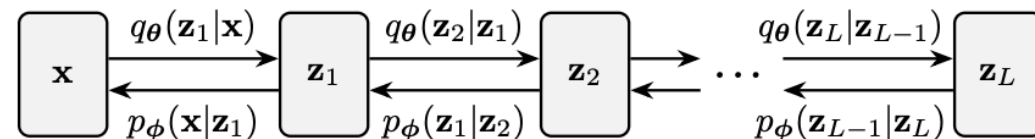


Figure 2.2: Computation graph of the HVAE. It has a hierarchical structure with stacked, trainable encoders and decoders across multiple latent layers.



Abstraction: Progressively finer levels (coarse $\rightarrow$ fine)

Sample $\mathbf{z}_L$ from prior, sequentially decoded down $[\mathbf{z}_L \rightarrow \mathbf{z}_{L-1} \rightarrow \dots \rightarrow \mathbf{z}_1]$, finally to the final observation $\mathbf{x}$

HAVE's Characteristics

⚙️ HVAE ELBO Derivation

$$\begin{aligned}
 \log p_{\text{HVAE}}(\mathbf{x}) &= \log \int p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L}) d\mathbf{z}_{1:L} \\
 &= \log \int \frac{p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L})}{q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x})} q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x}) d\mathbf{z}_{1:L} \\
 &= \log \mathbb{E}_{q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x})} \left[\frac{p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L})}{q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x})} \right] \\
 &\geq \mathbb{E}_{q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x})} \left[\log \frac{p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L})}{q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x})} \right] \\
 &=: \mathcal{L}_{\text{ELBO}}(\phi).
 \end{aligned}$$

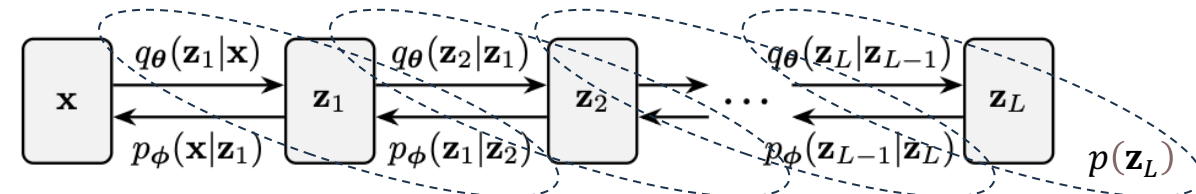
[2.1.3]

$$\begin{aligned}
 \mathcal{L}_{\text{ELBO}} &= \mathbb{E}_{q_{\theta}(\mathbf{z}_{1:L}|\mathbf{x})} \left[\log \frac{p(\mathbf{z}_L) \prod_{i=2}^L p_{\phi}(\mathbf{z}_{i-1}|\mathbf{z}_i) p_{\phi}(\mathbf{x}|\mathbf{z}_1)}{q_{\theta}(\mathbf{z}_1|\mathbf{x}) \prod_{i=2}^L q_{\theta}(\mathbf{z}_i|\mathbf{z}_{i-1})} \right] \\
 &= \mathbb{E}_q[\log p_{\phi}(\mathbf{x}|\mathbf{z}_1)] - \mathbb{E}_q[\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z}_1|\mathbf{x})||p_{\phi}(\mathbf{z}_1|\mathbf{z}_2))] \\
 &\quad - \sum_{i=2}^{L-1} \mathbb{E}_q[\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z}_i|\mathbf{z}_{i-1})||p_{\phi}(\mathbf{z}_i|\mathbf{z}_{i+1}))] \\
 &\quad - \mathbb{E}_q[\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z}_L|\mathbf{z}_{L-1})||p(\mathbf{z}_L))].
 \end{aligned}$$

- This spreads the information penalty across multiple levels and localizes learning signals via adjacent KL terms.
- These benefits arise from the hierarchical latent structure itself, not merely from making a flat VAE deeper.



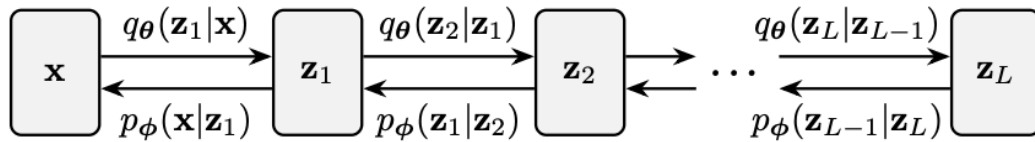
Each inference conditional is aligned with its top-down generative counterpart



HAVE's Impact & Challenge

⚙️ HVAE Impact

- Stacked layers enable progressive generation from coarse to fine details.



- This hierarchy simplifies modeling **complex high-dimensional data**.
- It is **a key principle behind modern generative models** like score-based methods and normalizing flows.

⚡ HAVE Challenge

- Lower layers dominate reconstruction, leaving higher latents under-utilized.
- Gradients to deep latents are indirectly learned and weak.



⚡ So, Diffusion Model...

- So, **diffusion models** retain hierarchy while avoiding these issues.
- Fixing encoding and learning only the reverse process improves stability and generation quality

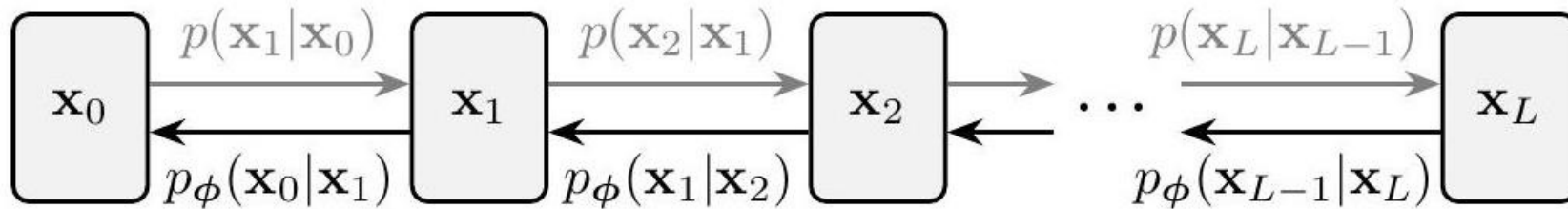
Variational Perspective: Denoising Diffusion Probabilistic Models (DDPM)

DDPM Overview and Core Concepts

⌘ Denoising Diffusion Probabilistic Models (DDPMs)

It represent a cornerstone of diffusion modeling.

They operate within a variational framework, much like VAEs and HVAEs, but with a clever twist.



➔ Forward Pass

Fixed Encoder: Gradually corrupts data by injecting Gaussian noise

Fixed transition kernel: $p(\mathbf{x}_i | \mathbf{x}_{i-1})$

Data evolves into an isotropic (모든 차원의 분산이 동일)
Gaussian distribution (i.e., $\mathcal{N}(\mu, \sigma^2 \mathbf{I})$)

⬅ Reverse Denoising Process

Learnable Decoder: Neural network learns to reverse noise corruption

Parameterized distribution: $p_\phi(\mathbf{x}_{i-1} | \mathbf{x}_i)$

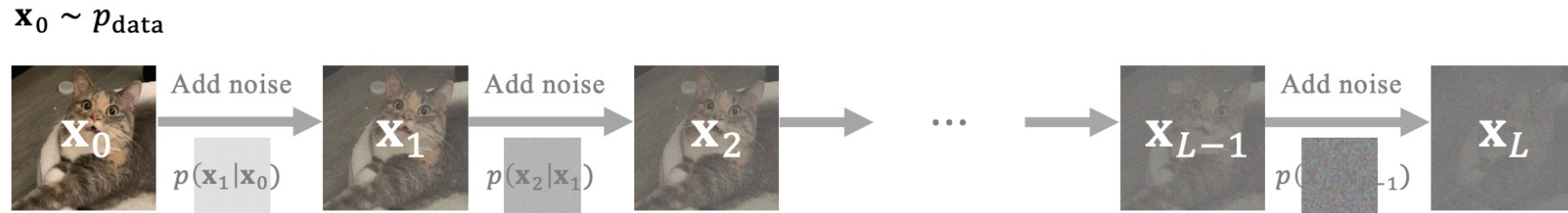
Each denoising step is a manageable task

DDPM – Forward Process (Fixed Encoder)

Forward Process – Fixed Encoder

The forward process is a fixed, non-trainable operation that serves as an encoder

It progressively corrupts data by adding noise, transforming it into a simple prior distribution $p_{prior} := \mathcal{N}(0, I)$



Fixed Gaussian Transitions

$$p(\mathbf{x}_i | \mathbf{x}_{i-1}) := \mathcal{N}\left(\mathbf{x}_i; \sqrt{1 - \beta_i^2} \mathbf{x}_{i-1}, \beta_i^2 \mathbf{I}\right)$$

- Process begins with $\mathbf{x}_0$, representing a sample drawn from the real data distribution p_{data}
- Sequence $\{\beta_i\}_{i=1}^L$ denotes a pre-determined, monotonically increasing noise schedule, where each $\beta_i \in (0,1)$ controls the variance injected at step i



Iterative Update $\alpha_i := \sqrt{1 - \beta_i^2}$

$$\mathbf{x}_i = \alpha_i \mathbf{x}_{i-1} + \beta_i \epsilon_i \quad \text{where } \epsilon_i \sim \mathcal{N}(0, I)$$

- Reparameterization trick
- At each step i , we scale down the previous state $\mathbf{x}_{i-1}$ by α_i and add a controlled amount of Gaussian noise scaled by β_i

Forward Process – Perturbation Kernel and Prior Distribution

⚙️ Direct Sampling at Step i

- a closed-form expression for the distribution of noisy samples at step i given the original data $\mathbf{x}_0$

$$p_i(\mathbf{x}_i | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_i; \bar{\alpha}_i \mathbf{x}_0, (1 - \bar{\alpha}_i^2) \mathbf{I})$$

where

$$\bar{\alpha}_i := \prod_{k=1}^i \sqrt{1 - \beta_k^2} = \prod_{k=1}^i \alpha_k.$$

- This means we can sample $\mathbf{x}_i$ directly from $\mathbf{x}_0$ as

$$\mathbf{x}_i = \bar{\alpha}_i \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_i^2} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$



🔄 Converge to Prior distribution

- Let the noise schedule $\{\beta_i\}_{i=1}^L$ be an increasing sequence, then the marginal distribution of the forward process converges as

$$p_L(\mathbf{x}_L | \mathbf{x}_0) \rightarrow \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ as } L \rightarrow \infty$$

- It motivates the choice of the prior distribution as

$$p_{\text{prior}} := \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Forward Process – Perturbation Kernel and Prior Distribution

1 한 줄 정의부터

α_i 와 β_i 는

forward diffusion 과정에서 "신호(signal)"와 "노이즈(noise)"의 비율을 정하는 스케줄 파라미터야.

- α_i : "이전 그림을 얼마나 덜 흐리게 남길지"
- β_i : "이번 단계에서 물감을 얼마나 더 흩뿌릴지"

뒤로 갈수록

- $\alpha_i \downarrow$
- $\beta_i \uparrow$

그래서 점점 완전한 노이즈가 됨.

2 Forward diffusion에서의 정의

DDPM의 forward process는 이렇게 정의돼:

$$x_i = \alpha_i x_{i-1} + \beta_i \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, I)$$

즉,

- α_i : 이전 상태를 얼마나 보존할지
- β_i : 새 노이즈를 얼마나 추가할지

를 결정하는 계수야.

3 왜 $\alpha_i^2 + \beta_i^2 = 1$ 인가?

보통 DDPM에서는

$$\alpha_i^2 = 1 - \beta_i^2$$

로 정의함. 이유는:

- 분산이 폭주하지 않게 하려고
- 각 step이 "정규화된 가우시안 노이즈 추가"가 되도록

즉, 에너지(분산)를 안정적으로 유지하기 위한 설계야.

4 $\bar{\alpha}_i$ 의 의미 (누적 신호량)

$$\bar{\alpha}_i = \prod_{k=1}^i \alpha_k$$

👉 x_0 (clean data)가 x_i 에 얼마나 남아 있는지를 나타냄.

그래서 forward marginal은:

$$x_i = \bar{\alpha}_i x + \sqrt{1 - \bar{\alpha}_i^2} \varepsilon$$

이 형태가 나옴.

$p_i(\mathbf{x}_i \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_i; \bar{\alpha}_i \mathbf{x}_0, (1 - \bar{\alpha}_i^2) \mathbf{I}) \rightarrow$ 유도 과정 상세 설명

1. 기본 설정

재귀 관계식:

$$\mathbf{x}_i = \alpha_i \mathbf{x}_{i-1} + \beta_i \boldsymbol{\epsilon}_i, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

여기서 $\alpha_i = \sqrt{1 - \beta_i^2}$ 이고, 모든 $\boldsymbol{\epsilon}_i$ 는 독립적입니다.

2. 단계별 전개

Step 1: $\mathbf{x}_1$ 표현

$$\mathbf{x}_1 = \alpha_1 \mathbf{x}_0 + \beta_1 \boldsymbol{\epsilon}_1$$

분포로 쓰면:

$$\mathbf{x}_1 \mid \mathbf{x}_0 \sim \mathcal{N}(\alpha_1 \mathbf{x}_0, \beta_1^2 \mathbf{I})$$

Step 2: $\mathbf{x}_2$ 표현

$$\mathbf{x}_2 = \alpha_2 \mathbf{x}_1 + \beta_2 \boldsymbol{\epsilon}_2$$

$$\mathbf{x}_2 = \alpha_2 (\alpha_1 \mathbf{x}_0 + \beta_1 \boldsymbol{\epsilon}_1) + \beta_2 \boldsymbol{\epsilon}_2$$

$$\mathbf{x}_2 = \alpha_2 \alpha_1 \mathbf{x}_0 + \alpha_2 \beta_1 \boldsymbol{\epsilon}_1 + \beta_2 \boldsymbol{\epsilon}_2$$

이제 가우시안 노이즈 항들을 하나로 합쳐야 합니다.

핵심: 독립 가우시안의 선형 결합

$\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2$ 가 독립이므로:

- $\alpha_2 \beta_1 \boldsymbol{\epsilon}_1 \sim \mathcal{N}(\mathbf{0}, (\alpha_2 \beta_1)^2 \mathbf{I})$
- $\beta_2 \boldsymbol{\epsilon}_2 \sim \mathcal{N}(\mathbf{0}, \beta_2^2 \mathbf{I})$

독립적인 가우시안 변수의 합:

$$\alpha_2 \beta_1 \boldsymbol{\epsilon}_1 + \beta_2 \boldsymbol{\epsilon}_2 \sim \mathcal{N}(\mathbf{0}, [(\alpha_2 \beta_1)^2 + \beta_2^2] \mathbf{I})$$

분산을 계산하면:

$$\text{Var} = (\alpha_2 \beta_1)^2 + \beta_2^2 = \alpha_2^2 \beta_1^2 + \beta_2^2$$

3. 분산 간소화의 핵심 트릭

$\alpha_i = \sqrt{1 - \beta_i^2}$ 라는 정의를 사용하면:

$$\alpha_i^2 = 1 - \beta_i^2 \quad \Rightarrow \quad \beta_i^2 = 1 - \alpha_i^2$$

따라서:

$$\text{Var} = \alpha_2^2 \beta_1^2 + \beta_2^2$$

$$= \alpha_2^2 (1 - \alpha_1^2) + (1 - \alpha_2^2)$$

$$= \alpha_2^2 - \alpha_2^2 \alpha_1^2 + 1 - \alpha_2^2$$

$$= 1 - \alpha_2^2 \alpha_1^2$$

$$= 1 - (\alpha_1 \alpha_2)^2$$

결과:

$$\mathbf{x}_2 = \alpha_1 \alpha_2 \mathbf{x}_0 + \sqrt{1 - (\alpha_1 \alpha_2)^2} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

따라서:

$$\mathbf{x}_2 \mid \mathbf{x}_0 \sim \mathcal{N}((\alpha_1 \alpha_2) \mathbf{x}_0, [1 - (\alpha_1 \alpha_2)^2] \mathbf{I})$$

$$p_i(\mathbf{x}_i | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_i; \bar{\alpha}_i \mathbf{x}_0, (1 - \bar{\alpha}_i^2) \mathbf{I}) \rightarrow \text{유도 과정 상세 설명}$$

Step 3: $\mathbf{x}_3$ 표현

같은 방식으로:

$$\begin{aligned}\mathbf{x}_3 &= \alpha_3 \mathbf{x}_2 + \beta_3 \epsilon_3 \\ &= \alpha_3 (\alpha_1 \alpha_2 \mathbf{x}_0 + \sqrt{1 - (\alpha_1 \alpha_2)^2} \epsilon) + \beta_3 \epsilon_3 \\ &= \alpha_1 \alpha_2 \alpha_3 \mathbf{x}_0 + \alpha_3 \sqrt{1 - (\alpha_1 \alpha_2)^2} \epsilon + \beta_3 \epsilon_3\end{aligned}$$

노이즈 항들의 분산:

$$\begin{aligned}\text{Var} &= \alpha_3^2 [1 - (\alpha_1 \alpha_2)^2] + \beta_3^2 \\ &= \alpha_3^2 [1 - (\alpha_1 \alpha_2)^2] + (1 - \alpha_3^2) \\ &= \alpha_3^2 - \alpha_3^2 (\alpha_1 \alpha_2)^2 + 1 - \alpha_3^2 \\ &= 1 - (\alpha_1 \alpha_2 \alpha_3)^2\end{aligned}$$



⚙️ Direct Sampling at Step i

- a closed-form expression for the distribution of noisy samples at step i given the original data $\mathbf{x}_0$

$$p_i(\mathbf{x}_i | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_i; \bar{\alpha}_i \mathbf{x}_0, (1 - \bar{\alpha}_i^2) \mathbf{I})$$

where

$$\bar{\alpha}_i := \prod_{k=1}^i \sqrt{1 - \beta_k^2} = \prod_{k=1}^i \alpha_k.$$

- This means we can sample $\mathbf{x}_i$ directly from $\mathbf{x}_0$ as

$$\mathbf{x}_i = \bar{\alpha}_i \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_i^2} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad \boxed{[2.2.1]}$$

$p_i(\mathbf{x}_i | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_i; \bar{\alpha}_i \mathbf{x}_0, (1 - \bar{\alpha}_i^2) \mathbf{I}) \rightarrow$ 유도 과정 상세 설명

4. 직접 샘플링 공식 (2.2.1)

위 결과로부터 임의의 timestep i 에서 직접 샘플링 가능:

$$\mathbf{x}_i = \bar{\alpha}_i \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_i^2} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

이것이 중요한 이유:

- 중간 스텝들을 거치지 않고 바로 $\mathbf{x}_0$ 에서 $\mathbf{x}_i$ 를 생성 가능!
- 학습 시 효율성 크게 향상

구체적 예시:

β 스케줄이 다음과 같다고 가정:

- $\beta_1 = 0.1, \beta_2 = 0.2, \beta_3 = 0.3$

그러면:

- $\alpha_1 = \sqrt{1 - 0.01} = 0.995$
- $\alpha_2 = \sqrt{1 - 0.04} = 0.980$
- $\alpha_3 = \sqrt{1 - 0.09} = 0.954$

$\mathbf{x}_3$ 을 직접 샘플링하려면:

$$\bar{\alpha}_3 = 0.995 \times 0.980 \times 0.954 \approx 0.930$$

$$\mathbf{x}_3 = 0.930 \cdot \mathbf{x}_0 + \sqrt{1 - 0.930^2} \boldsymbol{\epsilon} = 0.930 \cdot \mathbf{x}_0 + 0.367 \boldsymbol{\epsilon}$$

해석: 원본의 93%를 유지하고, 표준편차 0.367의 노이즈를 추가

5. Prior Distribution으로의 수렴

노이즈 스케줄 $\{\beta_i\}_{i=1}^L$ 이 증가하는 시퀀스이면:

$$p_L(\mathbf{x}_L | \mathbf{x}_0) \longrightarrow \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad \text{as } L \rightarrow \infty$$

왜냐하면:

- L 이 커질수록 $\bar{\alpha}_L = \prod_{k=1}^L \alpha_k \rightarrow 0$
- 따라서 $\mathbf{x}_L \approx \sqrt{1 - 0^2} \boldsymbol{\epsilon} = \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

이것이 **prior distribution**을 $p_{\text{prior}} := \mathcal{N}(\mathbf{0}, \mathbf{I})$ 로 선택하는 이유입니다!